

```
In [1]: import json, os, random, copy
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import Subset, TensorDataset, DataLoader
from sklearn.cluster import KMeans, MiniBatchKMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

cwd = os.getcwd()
project_root = os.path.dirname(cwd) if os.path.basename(cwd) == 'src' else c
results_dir = os.path.join(project_root, 'results')
os.makedirs(results_dir, exist_ok=True)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

config = {
    'dataset': {'root': './data', 'mean': [0.4914, 0.4822, 0.4465], 'std': [
    'feature_extractor': {'batch_size': 128},
    'classifier': {'test_batch_size': 128},
    'typiclust': {'random_state': 42, 'max_clusters': 500, 'kmeans_n_init':
    'evaluation': {'budgets': [10, 50, 100]},
    'visualization': {'subset_size': 2000, 'random_state': 42, 'num_clusters
}

transform_test = transforms.Compose([transforms.ToTensor(), transforms.Norma
transform_sup = transforms.Compose([transforms.RandomCrop(32, padding=4), tr

cifar10_full = torchvision.datasets.CIFAR10(root=config['dataset']['root'],
cifar10_sup_train = torchvision.datasets.CIFAR10(root=config['dataset']['roc
pool_loader = DataLoader(cifar10_full, batch_size=config['feature_extractor'
testset = torchvision.datasets.CIFAR10(root=config['dataset']['root'], trainr
test_loader = DataLoader(testset, batch_size=config['classifier']['test_batch

100%|██████████| 170M/170M [02:04<00:00, 1.37MB/s]
```

```
In [2]: class SimCLR_ResNet18(nn.Module):
    def __init__(self):
        super(SimCLR_ResNet18, self).__init__()
        base_model = torchvision.models.resnet18(weights=None)
        base_model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding
        base_model.maxpool = nn.Identity()
        self.encoder = nn.Sequential(*list(base_model.children())[:-1])
        self.projector = nn.Sequential(nn.Linear(512, 512, bias=False), nn.L
    def forward(self, x):
        h = self.encoder(x).view(x.size(0), -1)
        z = self.projector(h)
        return h, z

model = SimCLR_ResNet18().to(device)
```

```

checkpoint_path = os.path.join(project_root, 'models', 'simclr_resnet18_epoch')
if os.path.exists(checkpoint_path):
    model.load_state_dict(torch.load(checkpoint_path, map_location=device))
    print("Loaded checkpoint.")

model.eval()
embeddings, true_labels = [], []
with torch.no_grad():
    for images, labels in pool_loader:
        features, _ = model(images.to(device))
        embeddings.append(F.normalize(features, p=2, dim=1).cpu().numpy())
        true_labels.append(labels.numpy())
embeddings = np.concatenate(embeddings, axis=0)
true_labels = np.concatenate(true_labels, axis=0)

test_embeddings, test_labels = [], []
with torch.no_grad():
    for images, labels in test_loader:
        features, _ = model(images.to(device))
        test_embeddings.append(F.normalize(features, p=2, dim=1).cpu().numpy())
        test_labels.append(labels.numpy())
test_embeddings = np.concatenate(test_embeddings, axis=0)
test_labels = np.concatenate(test_labels, axis=0)
test_dl = DataLoader(TensorDataset(torch.tensor(test_embeddings, dtype=torch.float32), torch.tensor(test_labels, dtype=torch.long)))

```

Loaded checkpoint.

```

In [3]: def get_typiclust_queries(embeddings, B, max_clusters=500, labeled_idx=None):
    labeled_idx = labeled_idx if labeled_idx is not None else []
    N, K = len(embeddings), min(len(labeled_idx) + B, max_clusters)
    kmeans = (KMeans if K <= 50 else MiniBatchKMeans)(n_clusters=K, random_state=None)
    cluster_labels = kmeans.fit_predict(embeddings)
    clusters = {i: [] for i in range(K)}
    for idx, label in enumerate(cluster_labels): clusters[label].append(idx)
    cluster_labeled_counts = {i: 0 for i in range(K)}
    for idx in labeled_idx: cluster_labeled_counts[cluster_labels[idx]] += 1
    queries = []
    mask = np.zeros(N, dtype=bool)
    for idx in labeled_idx: mask[idx] = True
    for _ in range(B):
        valid = [i for i in range(K) if len(clusters[i]) >= 5]
        if not valid: valid = list(range(K))
        min_l = min([cluster_labeled_counts[i] for i in valid])
        best_c = max([i for i in valid if cluster_labeled_counts[i] == min_l])
        indices = np.array(clusters[best_c])
        unlabeled = indices[~mask[indices]]
        if len(unlabeled) == 0: break
        nn = NearestNeighbors(n_neighbors=min(20, len(indices))).fit(embeddings[indices])
        dist, _ = nn.kneighbors(embeddings[unlabeled])
        typicality = 1.0 / (np.mean(dist, axis=1) + 1e-8)
        best_p = unlabeled[np.argmax(typicality)]
        queries.append(best_p); mask[best_p] = True; cluster_labeled_counts[cluster_labels[best_p]] += 1
    return queries

def get_entropy_queries(embeddings, labels, B):
    idx = np.random.choice(len(embeddings), min(10, B), replace=False)

```

```

if B <= 10: return idx.tolist()
clf = nn.Linear(embeddings.shape[1], 10).to(device)
opt = optim.SGD(clf.parameters(), lr=0.1)
for _ in range(20):
    opt.zero_grad(); nn.CrossEntropyLoss()(clf(torch.tensor(embeddings[i])))
with torch.no_grad():
    p = F.softmax(clf(torch.tensor(embeddings).to(device)), dim=1)
    ent = -torch.sum(p * torch.log(p + 1e-8), dim=1).cpu().numpy()
return np.argsort(ent)[-B:].tolist()

```

In [4]:

```

class BasicBlock(nn.Module):
    def __init__(self, in_p, out_p, stride, drop=0.0, act_before=False):
        super().__init__()
        self.bn1, self.relu1 = nn.BatchNorm2d(in_p), nn.LeakyReLU(0.1, True)
        self.conv1 = nn.Conv2d(in_p, out_p, 3, stride, 1, bias=False)
        self.bn2, self.relu2 = nn.BatchNorm2d(out_p), nn.LeakyReLU(0.1, True)
        self.conv2 = nn.Conv2d(out_p, out_p, 3, 1, 1, bias=False)
        self.drop, self.equal = drop, in_p == out_p
        self.shortcut = None if self.equal else nn.Conv2d(in_p, out_p, 1, stride, 0, bias=False)
        self.act_before = act_before
    def forward(self, x):
        if not self.equal and self.act_before:
            out = self.relu1(self.bn1(x))
        else:
            out = self.relu1(self.bn1(x))
        out = self.relu2(self.bn2(self.conv1(out if self.equal else out)))
        if self.drop > 0: out = F.dropout(out, p=self.drop, training=self.training)
        return (x if self.equal else self.shortcut(x)) + self.conv2(out)

class WideResNet(nn.Module):
    def __init__(self, depth=28, num_c=10, widen=2, drop=0.0):
        super().__init__()
        nC = [16, 16*widen, 32*widen, 64*widen]; n = (depth - 4) // 6
        self.conv1 = nn.Conv2d(3, nC[0], 3, 1, 1, bias=False)
        self.block1 = nn.Sequential(*[BasicBlock(nC[0] if i==0 else nC[1], nC[1], 1, drop) for i in range(n)])
        self.block2 = nn.Sequential(*[BasicBlock(nC[1] if i==0 else nC[2], nC[2], 1, drop) for i in range(n)])
        self.block3 = nn.Sequential(*[BasicBlock(nC[2] if i==0 else nC[3], nC[3], 1, drop) for i in range(n)])
        self.bn1, self.relu, self.fc = nn.BatchNorm2d(nC[3]), nn.LeakyReLU(0.1, True), nn.Linear(nC[3], num_c)
    def forward(self, x):
        out = self.relu(self.bn1(self.block3(self.block2(self.block1(self.conv1(x)))))
        return self.fc(F.avg_pool2d(out, 8).view(x.size(0), -1))

def eval_fully_sup(q):
    clf = torchvision.models.resnet18(weights=None)
    clf.conv1 = nn.Conv2d(3, 64, 3, 1, 1, bias=False); clf.maxpool = nn.Identity()
    clf = clf.to(device); opt = optim.SGD(clf.parameters(), 0.025, 0.9, nestor=optim.Nesterov)
    dl = DataLoader(Subset(cifar10_sup_train, q), batch_size=min(len(q), 32))
    for _ in range(200):
        clf.train()
        for x, y in dl: opt.zero_grad(); nn.CrossEntropyLoss()(clf(x.to(device)))
        sched.step()
    clf.eval(); c = 0
    with torch.no_grad():
        for x, y in test_loader: c += (clf(x.to(device)).max(1)[1] == y.to(device)).sum().item()
    return 100 * c / len(testset)

```

```

def eval_self_sup(q):
    clf = nn.Linear(embeddings.shape[1], 10).to(device); opt = optim.SGD(clf.parameters())
    dl = DataLoader(TensorDataset(torch.tensor(embeddings[q]), torch.tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])), batch_size=1)
    for _ in range(5):
        clf.train()
        for x, y in dl: opt.zero_grad(); nn.CrossEntropyLoss()(clf(x.to(device)).to(device), y.to(device)).backward()
        sched.step()
    clf.eval(); c = 0
    with torch.no_grad():
        for x, y in test_dl: c += (clf(x.to(device)).max(1)[1] == y.to(device)).sum().item()
    return 100 * c / len(testset)

def eval_semi_sup(queries):
    from torchvision.transforms import RandAugment
    import torch
    import torch.nn as nn
    import torch.optim as optim
    import torch.nn.functional as F
    import torchvision
    from torch.utils.data import DataLoader, Subset

    weak = transforms.Compose([
        transforms.RandomCrop(32, 4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(config['dataset']['mean'], config['dataset']['std'])
    ])
    strong = transforms.Compose([
        transforms.RandomCrop(32, 4),
        transforms.RandomHorizontalFlip(),
        RandAugment(2, 10),
        transforms.ToTensor(),
        transforms.Normalize(config['dataset']['mean'], config['dataset']['std'])
    ])

    clf = WideResNet(28, 10, 2).to(device)
    opt = optim.SGD(clf.parameters(), lr=0.03, momentum=0.9, weight_decay=0.01)

    sup_ds = torchvision.datasets.CIFAR10(root=config['dataset']['root'], transform=transforms.Compose([transforms.ToTensor(), transforms.Normalize(config['dataset']['mean'], config['dataset']['std'])]), download=True)
    dl_l = DataLoader(Subset(sup_ds, queries), batch_size=64, shuffle=True)

    unlabeled = list(set(range(len(cifar10_full)) - set(queries)))
    u_ds_weak = torchvision.datasets.CIFAR10(root=config['dataset']['root'], transform=weak, download=True)
    u_ds_strong = torchvision.datasets.CIFAR10(root=config['dataset']['root'], transform=strong, download=True)
    dl_u_weak = DataLoader(Subset(u_ds_weak, unlabeled), batch_size=64, shuffle=True)
    dl_u_strong = DataLoader(Subset(u_ds_strong, unlabeled), batch_size=64, shuffle=True)

    class_counts = torch.zeros(10).to(device)
    base_thresh = 0.95

    NUM_ITERATIONS = 1000
    labeled_iter = iter(dl_l)
    weak_iter = iter(dl_u_weak)
    strong_iter = iter(dl_u_strong)

    for it in range(NUM_ITERATIONS):

```

```

clf.train()

try: x_l, y_l = next(labeled_iter)
except StopIteration: labeled_iter = iter(dl_l); x_l, y_l = next(lab

try: x_w, _ = next(weak_iter)
except StopIteration: weak_iter = iter(dl_u_weak); x_w, _ = next(wea

try: x_s, _ = next(strong_iter)
except StopIteration: strong_iter = iter(dl_u_strong); x_s, _ = next

x_l, y_l = x_l.to(device), y_l.to(device)
x_w, x_s = x_w.to(device), x_s.to(device)

sup_loss = nn.CrossEntropyLoss()(clf(x_l), y_l)

with torch.no_grad():
    probs_w = F.softmax(clf(x_w), dim=1)
    max_probs, pseudo_y = probs_w.max(1)

passed_base_mask = max_probs >= base_thresh
for c in range(10):
    class_counts[c] += (pseudo_y[passed_base_mask] == c).sum()

max_count = class_counts.max()
beta = class_counts / max_count if max_count > 0 else torch.zeros(10

per_class_thresh = beta[pseudo_y] * base_thresh
mask = max_probs >= per_class_thresh

if mask.sum() > 0:
    unsup_loss = nn.CrossEntropyLoss()(clf(x_s[mask]), pseudo_y[mask]
else:
    unsup_loss = torch.tensor(0.0).to(device)

loss = sup_loss + unsup_loss

opt.zero_grad()
loss.backward()
opt.step()

clf.eval()
correct = 0
with torch.no_grad():
    for x, y in test_loader:
        correct += (clf(x.to(device)).max(1)[1] == y.to(device)).sum().i

return 100 * correct / len(testset)

```

```

In [5]: budgets, res = [10, 20, 30, 40, 50, 60], {'fully_sup': {'tc': [], 'rand': []}
for b in budgets:
    print(f"Budget {b}")
    tc_q = get_typiclust_queries(embeddings, b, 500)
    rand_q = random.sample(range(len(embeddings)), b)
    for k, f in [('fully_sup', eval_fully_sup), ('self_sup', eval_self_sup)]:
        res[k]['tc'].append(f(tc_q)); res[k]['rand'].append(f(rand_q))

```

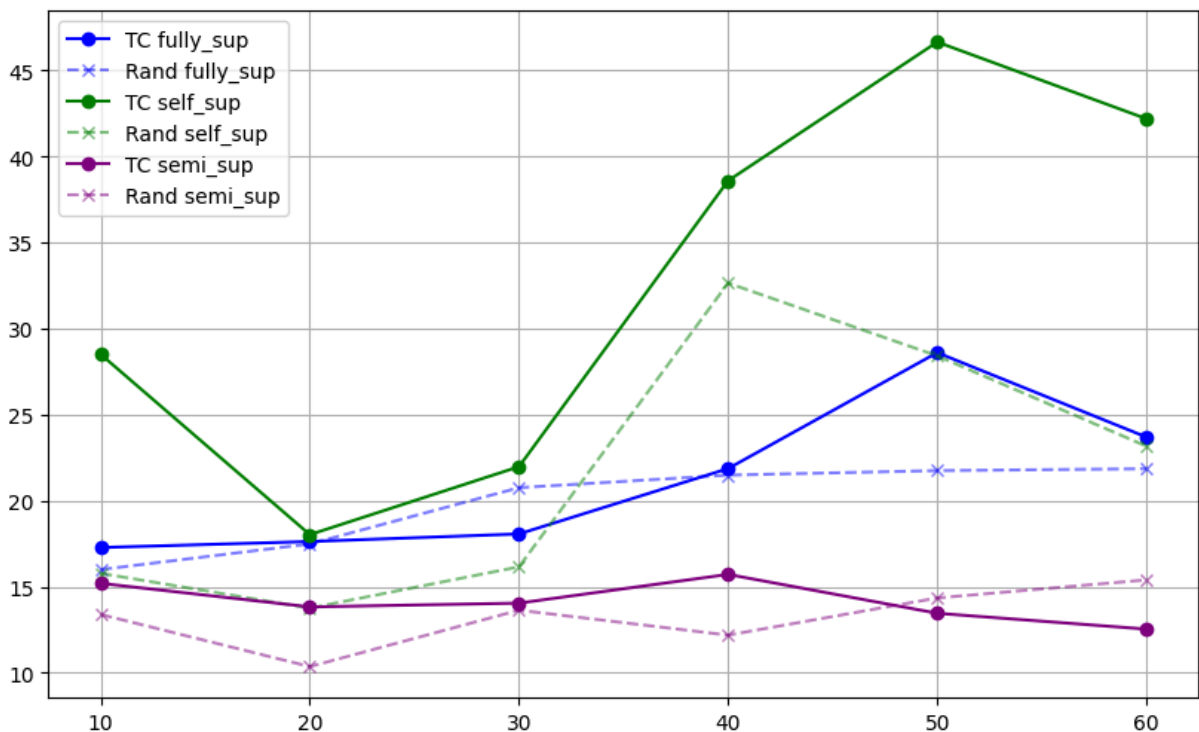
```

res['semi_sup']['tc'].append(np.mean([eval_semi_sup(tc_q) for _ in range
res['semi_sup']['rand'].append(np.mean([eval_semi_sup(rand_q) for _ in range

plt.figure(figsize=(10, 6))
c = {'fully_sup': 'blue', 'self_sup': 'green', 'semi_sup': 'purple'}
for k in res:
    plt.plot(budgets, res[k]['tc'], 'o-', label=f'TC {k}', color=c[k])
    plt.plot(budgets, res[k]['rand'], 'x--', label=f'Rand {k}', color=c[k],
plt.legend(); plt.grid(True); plt.savefig('combined_plot.pdf'); plt.show()

```

Budget 10
 Budget 20
 Budget 30
 Budget 40
 Budget 50
 Budget 60



```

In [9]: from sklearn.manifold import TSNE

subset_size = 15000
subset_idx = np.arange(15000)
emb_subset = embeddings[subset_idx]

tsne = TSNE(n_components=2, random_state=config['visualization']['random_state'])
emb_2d = tsne.fit_transform(emb_subset)

kmeans_viz = KMeans(n_clusters=config['visualization']['num_clusters_viz'],
cluster_labels_viz = kmeans_viz.fit_predict(emb_subset)

tc_queries_viz = get_typiclust_queries(emb_subset, B=10, max_clusters=config['visualization']['max_clusters_viz'])

plt.figure(figsize=(10, 8))
plt.scatter(emb_2d[:, 0], emb_2d[:, 1], c=cluster_labels_viz, cmap='tab20',
query_2d = emb_2d[tc_queries_viz]

```

```

plt.scatter(query_2d[:, 0], query_2d[:, 1], c='black', marker='x', s=150, li
plt.title('t-SNE Visualization of Feature Space and Selected Queries')
plt.xlabel('t-SNE Dimension 1')
plt.ylabel('t-SNE Dimension 2')
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(results_dir, config['visualization']['tsne_filename
plt.show()

```

